



Цифровые технологии управления
транспортными процессами

Сервис-ориентированное
программирование

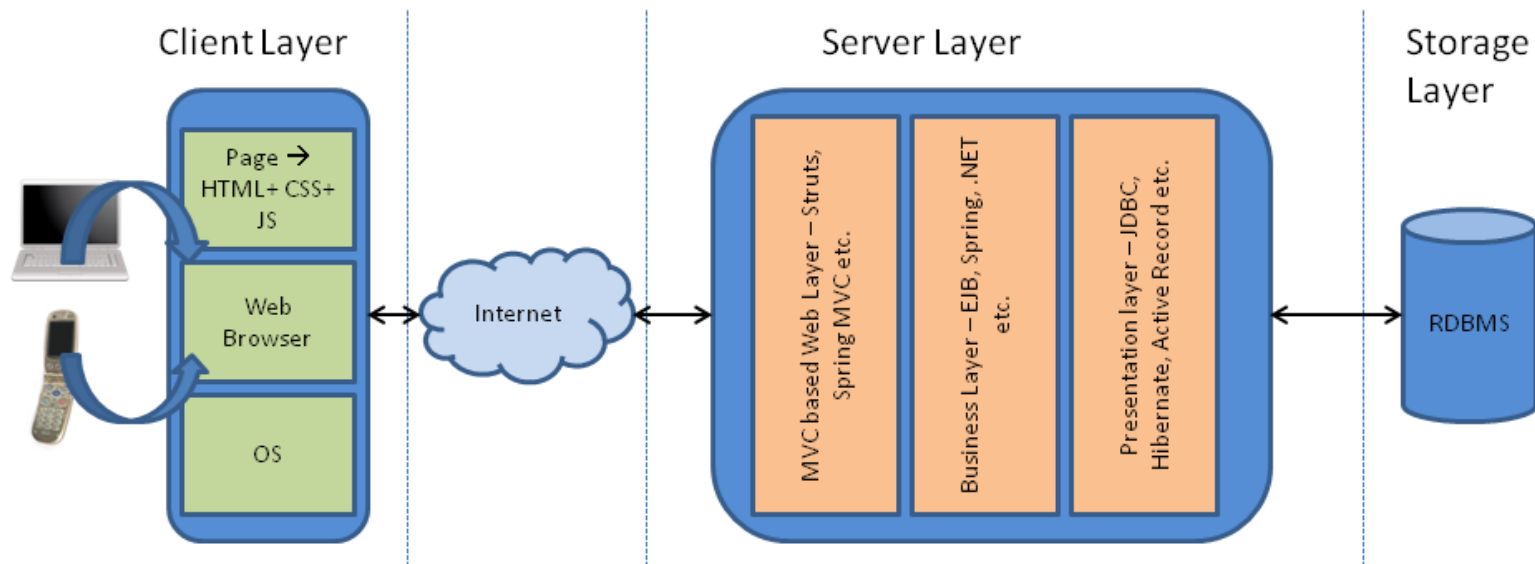
REST

Архитектура клиент-сервер



Веб-приложение

Веб-приложение – это клиент-серверное приложение, в котором клиентом выступает браузер, а сервером – веб-сервер.





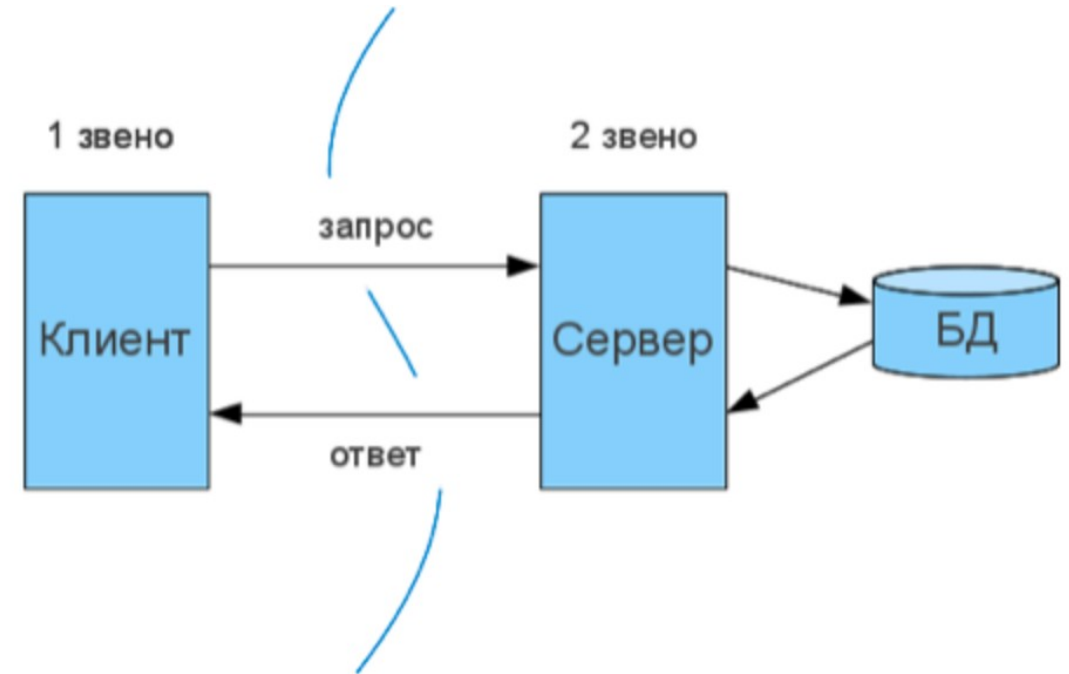
Архитектура «клиент-сервер»

- Архитектура «клиент-сервер» определяет общие принципы организации взаимодействия в сети, где имеются серверы, узлы-поставщики некоторых специфичных функций (сервисов) и клиенты (потребители этих функций).
- Практические реализации такой архитектуры называются клиент-серверными технологиями.



Двухзвенная архитектура

- Двухзвенная архитектура - распределение трех базовых компонентов между двумя узлами (клиентом и сервером).
- Двухзвенная архитектура используется в клиент-серверных системах, где сервер отвечает на клиентские запросы напрямую и в полном объеме.





Двухзвенная архитектура

- Расположение компонентов на стороне клиента или сервера определяет следующие основные модели их взаимодействия в рамках двухзвенной архитектуры:
 - **Сервер терминалов** — распределенное представление данных.
 - **Файл-сервер** — доступ к удаленной базе данных и файловым ресурсам.
 - **Сервер БД** — удаленное представление данных.
 - **Сервер приложений** — удаленное приложение.



Клиент

- Клиент – это браузер, но встречаются и исключения (в тех случаях, когда один веб-сервер (BC1) выполняет запрос к другому (BC2), роль клиента играет веб-сервер BC1).
- В классической ситуации (когда роль клиента выполняет браузер) для того, чтобы пользователь увидел графический интерфейс приложения в окне браузера, последний должен обработать полученный ответ веб-сервера, в котором будет содержаться информация, реализованная с применением HTML, CSS, JS (самые используемые технологии). Именно эти технологии «дают понять» браузеру, как именно необходимо «отрисовать» все, что он получил в ответе.



Веб-сервер

- Веб-сервер – это сервер, принимающий HTTP-запросы от клиентов и выдающий им HTTP-ответы.
- Веб-сервером называют как программное обеспечение, выполняющее функции веб-сервера, так и непосредственно компьютер, на котором это программное обеспечение работает. Наиболее распространенными видами ПО веб-серверов являются Apache, IIS и NGINX.
- На веб-сервере функционирует приложение, которое может быть реализовано с применением самых разнообразных языков программирования: PHP, Python, Ruby, Java, Perl, .NET и пр.

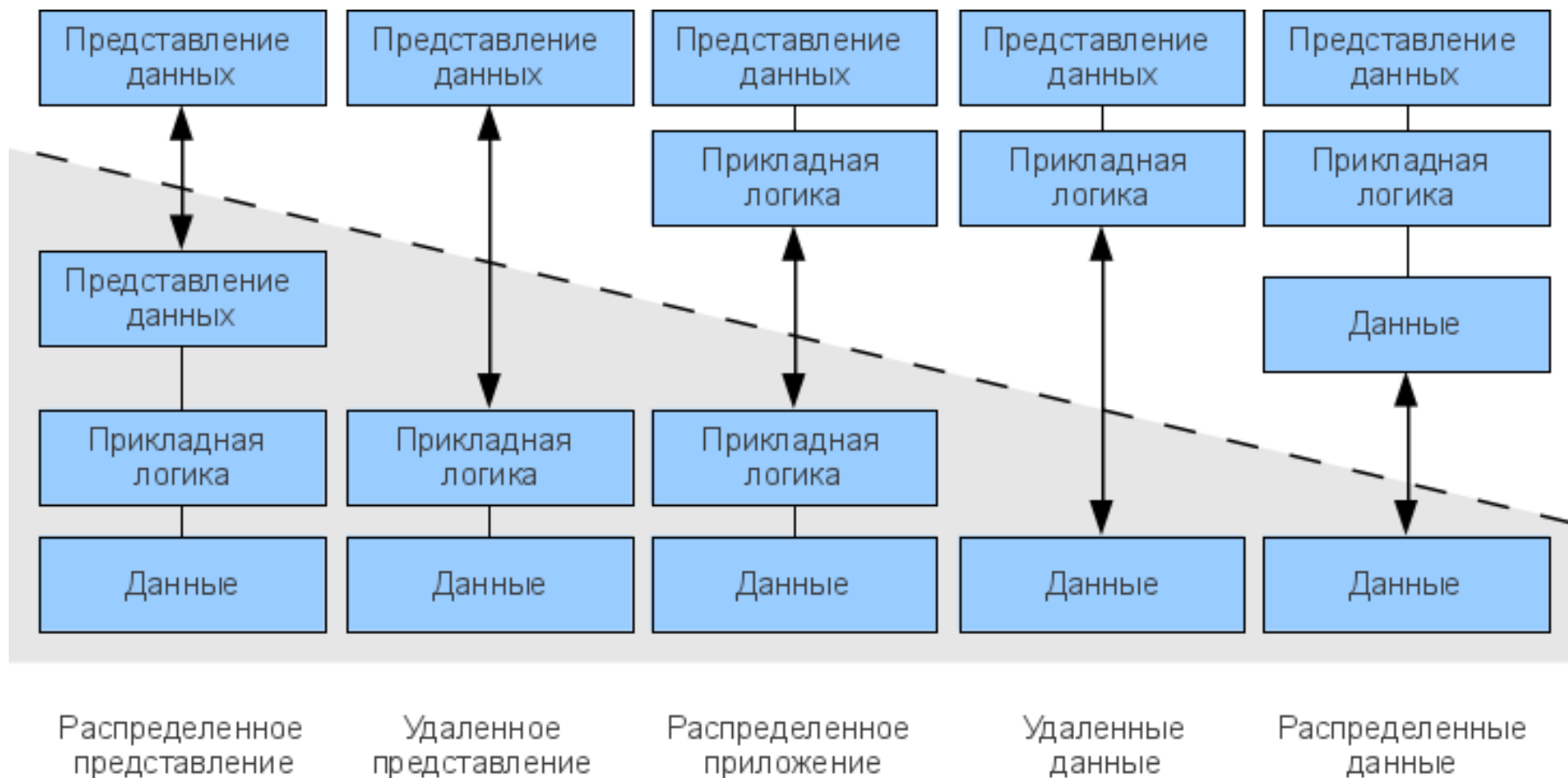


База данных

База данных фактически не является частью веб-сервера, но большинство приложений просто не могут выполнять все возложенные на них функции без нее, так как именно в базе данных хранится вся динамическая информация приложения (учетные, пользовательские данные и пр).



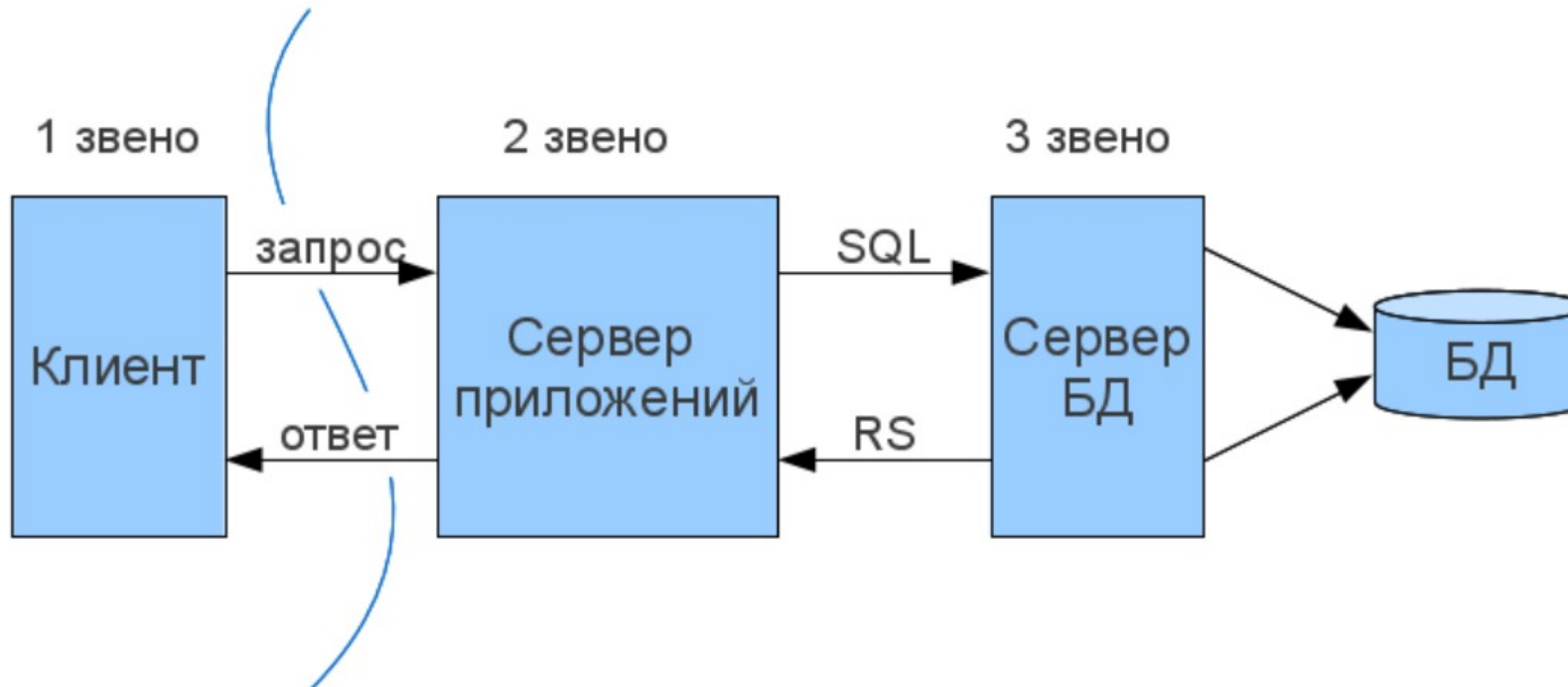
Модели клиент-серверного взаимодействия





Трехзвенная архитектура

Трехзвенная архитектура - сетевое приложение разделено на две и более частей, каждая из которых может выполняться на отдельном компьютере. Выделенные части приложения взаимодействуют друг с другом, обмениваясь сообщениями в заранее согласованном формате.





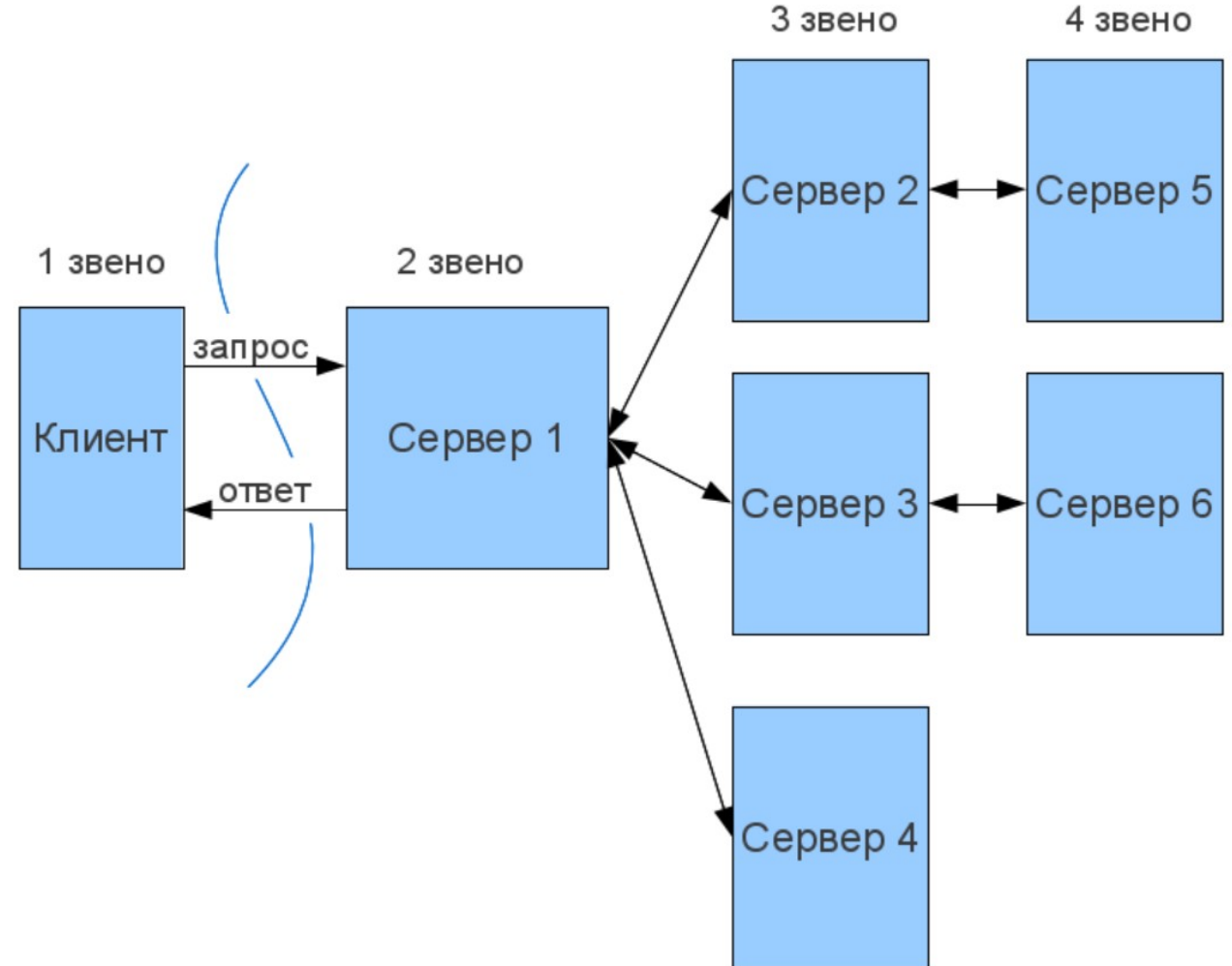
Трехзвенная архитектура

- Третьим звеном в трехзвенной архитектуре становится сервер приложений, т.е. компоненты распределяются следующим образом:
 - Представление данных — на стороне клиента.
 - Прикладной компонент — на выделенном сервере приложений (как вариант, выполняющем функции промежуточного ПО).
 - Управление ресурсами — на сервере БД, который и представляет запрашиваемые данные.



Multi-tier

Трехзвенная архитектура может быть расширена до многозвенной (N-tier, Multi-tier) путем выделения дополнительных серверов, каждый из которых будет представлять собственные сервисы и пользоваться услугами прочих серверов разного уровня.





2 VS 3

- Двухзвенная архитектура проще, так как все запросы обслуживаются одним сервером, но именно из-за этого она менее надежна и предъявляет повышенные требования к производительности сервера.
- Трехзвенная архитектура сложнее, но, благодаря тому, что функции распределены между серверами второго и третьего уровня, эта архитектура предоставляет:
 - Высокую степень гибкости и масштабируемости.
 - Высокую безопасность (т.к. защиту можно определить для каждого сервиса или уровня).
 - Высокую производительность (т.к. задачи распределены между серверами).



Цифровые технологии управления
транспортными процессами

Сервис-ориентированное
программирование

REST

Клиент-серверные технологии



Типы сервисов

Архитектура клиент-сервер применяется в большом числе сетевых технологий, используемых для доступа к различным сетевым сервисам.



Web-серверы

- Изначально предоставляли доступ к гипертекстовым документам по протоколу HTTP (Hyper Text Transfer Protocol).
- Сейчас поддерживают расширенные возможности, в частности, работу с бинарными файлами (изображения, мультимедиа и т.п.).



Серверы приложений

- Предназначены для централизованного решения прикладных задач в некоторой предметной области. Для этого пользователи имеют право запускать серверные программы на исполнение.
- Использование серверов приложений позволяет снизить требования к конфигурации клиентов и упрощает общее управление сетью.



Серверы баз данных

Серверы баз данных используются для обработки пользовательских запросов на языке SQL. При этом, СУБД находится на сервере, к которому и подключаются клиентские приложения.



Файл-серверы

Файл-сервер хранит информацию в виде файлов и предоставляет пользователям доступ к ней. Как правило, файл-сервер обеспечивает и определенный уровень защиты от несанкционированного доступа.



Прокси-сервер

- Во-первых, действует как посредник, помогая пользователям получить информацию из Интернета и, при этом, обеспечивая защиту сети.
- Во-вторых, сохраняет часто запрашиваемую информацию в кэш-памяти на локальном диске, быстро доставляя ее пользователям, без повторного обращения к Интернету.



Файрволы (брандмауэры)

Межсетевые экраны, анализирующие и фильтрующие проходящий сетевой трафик, с целью обеспечения безопасности сети.



Почтовые серверы

Предоставляют услуги по отправке и получению электронных почтовых сообщений.



Серверы удаленного доступа (RAS)

- Эти системы обеспечивают связь с сетью по коммутируемым линиям. Удаленный сотрудник может использовать ресурсы корпоративной ЛВС, подключившись к ней с помощью обычного модема.
- Для доступа к тем или иным сетевым сервисам используются клиенты, возможности которых характеризуются понятием «толщины». Оно определяет конфигурацию оборудования и программное обеспечение, имеющиеся у клиента.



«Тонкий» клиент

- Этот термин определяет клиента, вычислительных ресурсов которого достаточно лишь для запуска необходимого сетевого приложения через web-интерфейс. Пользовательский интерфейс такого приложения формируется средствами HTML + JavaScript, вся прикладная логика выполняется на сервере.
- Для работы тонкого клиента достаточно лишь обеспечить возможность запуска web-браузера, в окне которого и осуществляются все действия. По этой причине web-браузер часто называют «универсальным клиентом».



«Толстый» клиент

- Таковым является рабочая станция или персональный компьютер, работающие под управлением собственной дисковой операционной системы и имеющие необходимый набор программного обеспечения. К сетевым серверам «толстые» клиенты обращаются, в основном, за дополнительными услугами (например, доступ к web-серверу или корпоративной базе данных).
- Так же под «толстым» клиентом подразумевается и клиентское сетевое приложение, запущенное под управлением локальной ОС. Такое приложение совмещает компонент представления данных (графический пользовательский интерфейс ОС) и прикладной компонент (вычислительные мощности клиентского компьютера).



«rich»-клиент

- «Rich» -клиент, своего рода, компромисс между «толстым» и «тонким» клиентом. Как и «тонкий» клиент, «rich»-клиент также представляет графический интерфейс, описываемый уже средствами XML и включающий некоторую функциональность толстых клиентов (например, интерфейс drag-and-drop, вкладки, множественные окна, выпадающие меню и т.п.)
- Прикладная логика «rich»-клиента также реализована на сервере. Данные отправляются в стандартном формате обмена, на основе того же XML (протоколы SOAP, XML-RPC) и интерпретируются клиентом.



Протоколы «rich»-клиентов

- XAML (eXtensible Application Markup Language) — разработан Microsoft и используется в приложениях на платформе .NET.
- XUL (XML User Interface Language) — стандарт, разработанный в рамках проекта Mozilla, используется, например, в почтовом клиенте Mozilla Thunderbird или браузере Mozilla Firefox.
- Flex — мультимедийная технология на основе XML, разработанная Macromedia/Adobe.



Протокол передачи данных

- Протокол передачи данных — набор соглашений интерфейса логического уровня, которые определяют обмен данными между различными программами. Эти соглашения задают единообразный способ передачи сообщений и обработки ошибок при взаимодействии ПО.
- Сетевой протокол — набор правил и действий (очерёдности действий), позволяющий осуществлять соединение и обмен данными между двумя и более включёнными в сеть устройствами.



Сетевые протоколы

- TCP/IP — набор протоколов передачи данных, получивший название от двух принадлежащих ему протоколов: TCP (англ. Transmission Control Protocol) и IP (англ. Internet Protocol).
- Наиболее известные протоколы, рассмотренные ранее в дисциплинах:
 - HTTP (Hyper Text Transfer Protocol) — это протокол передачи гипертекста.
 - HTTPS (HyperText Transfer Protocol Secure) - расширение протокола HTTP для поддержки шифрования, в целях повышения безопасности. Данные в протоколе HTTPS передаются поверх криптографических протоколов SSL или TLS.
 - SSL (Secure Sockets Layer — уровень защищённых сокетов) — криптографический протокол, который подразумевает более безопасную связь.
 - FTP (File Transfer Protocol) — это протокол передачи файлов со специального файлового сервера на компьютер пользователя.
 - POP3 (Post Office Protocol) — это стандартный протокол почтового соединения.



Цифровые технологии управления
транспортными процессами

Сервис-ориентированное
программирование

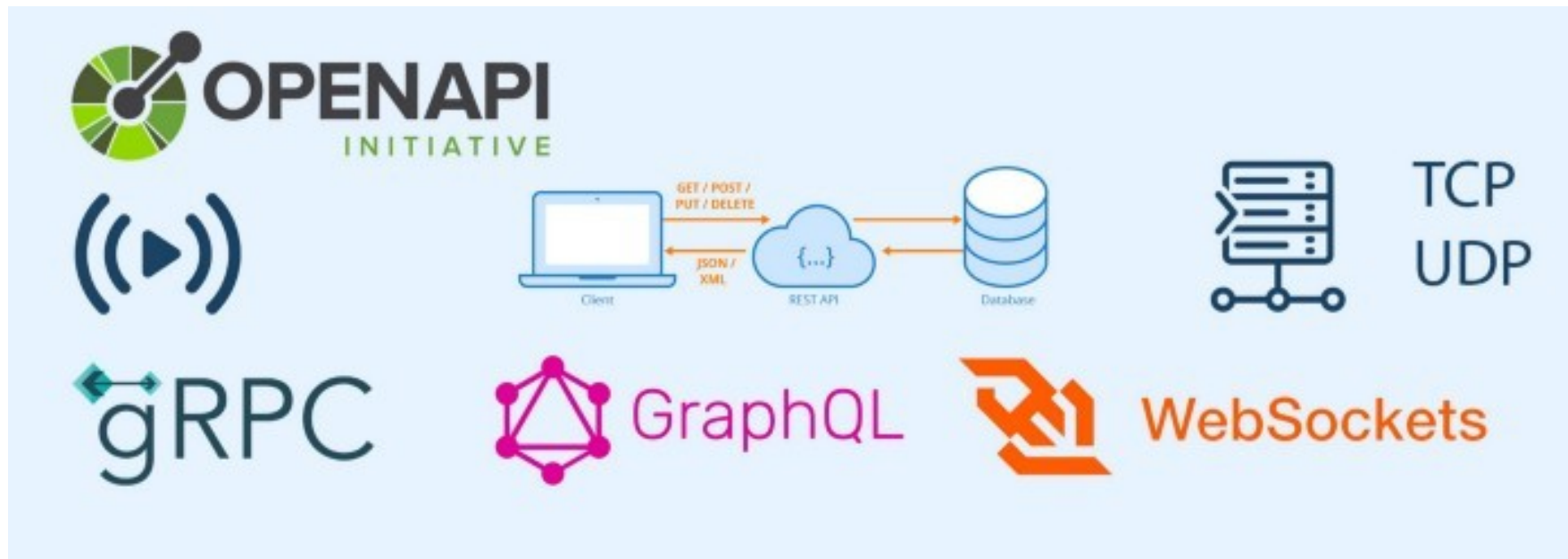
REST

НО...



Современный подход к клиент-серверной коммуникации

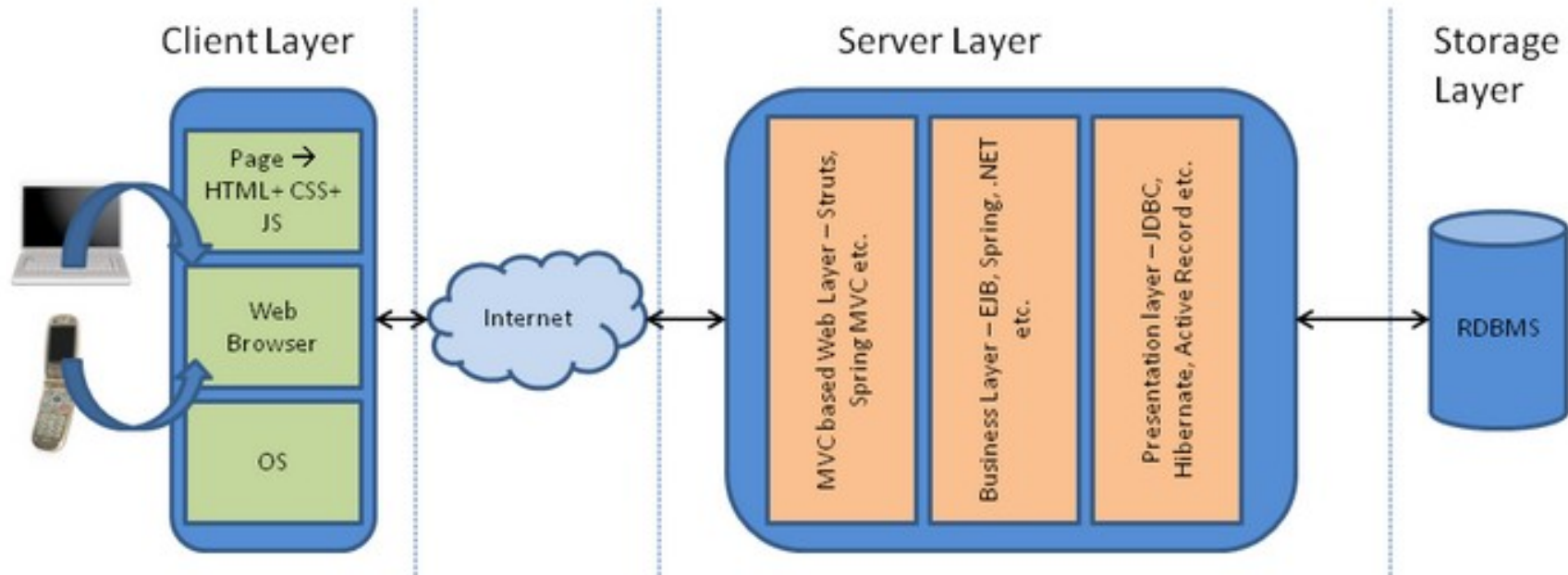
Рассмотрим современные подходы к коммуникации, как клиент-серверной, так и между сервисами и микросервисами.





Изменения архитектуры веб-приложений

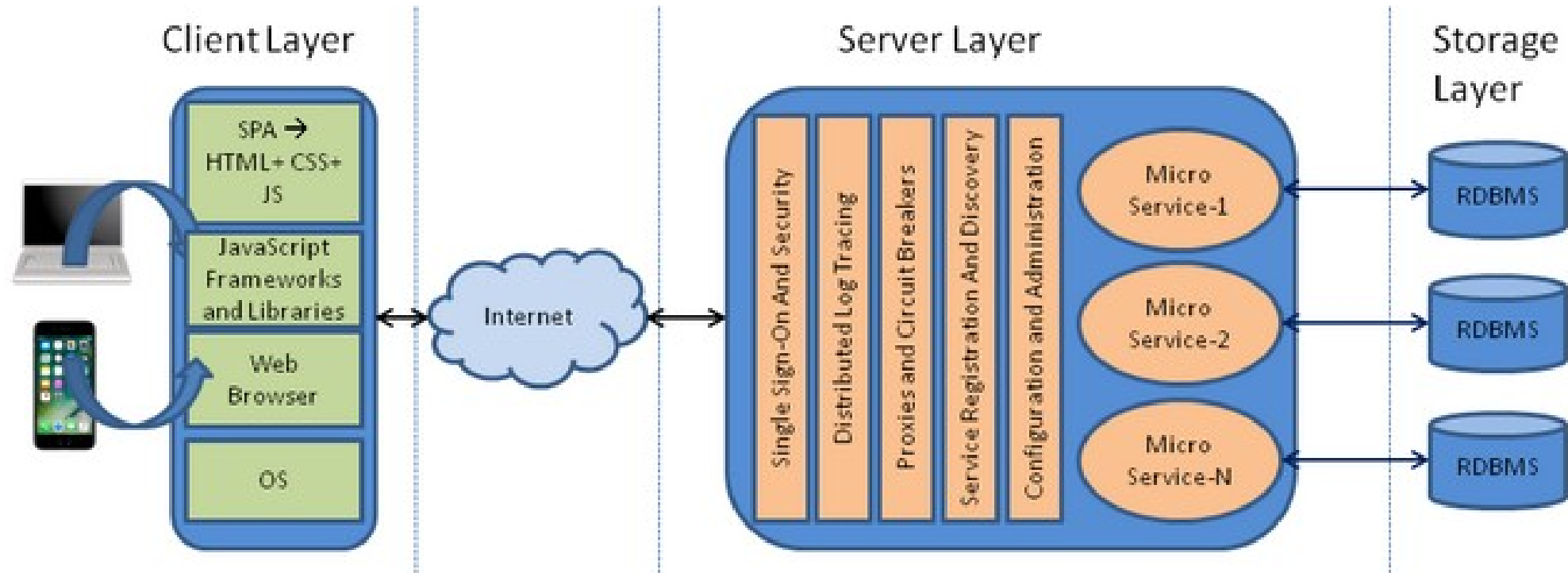
Рассмотрим традиционную N-Tier архитектуру:





Изменения архитектуры веб-приложений

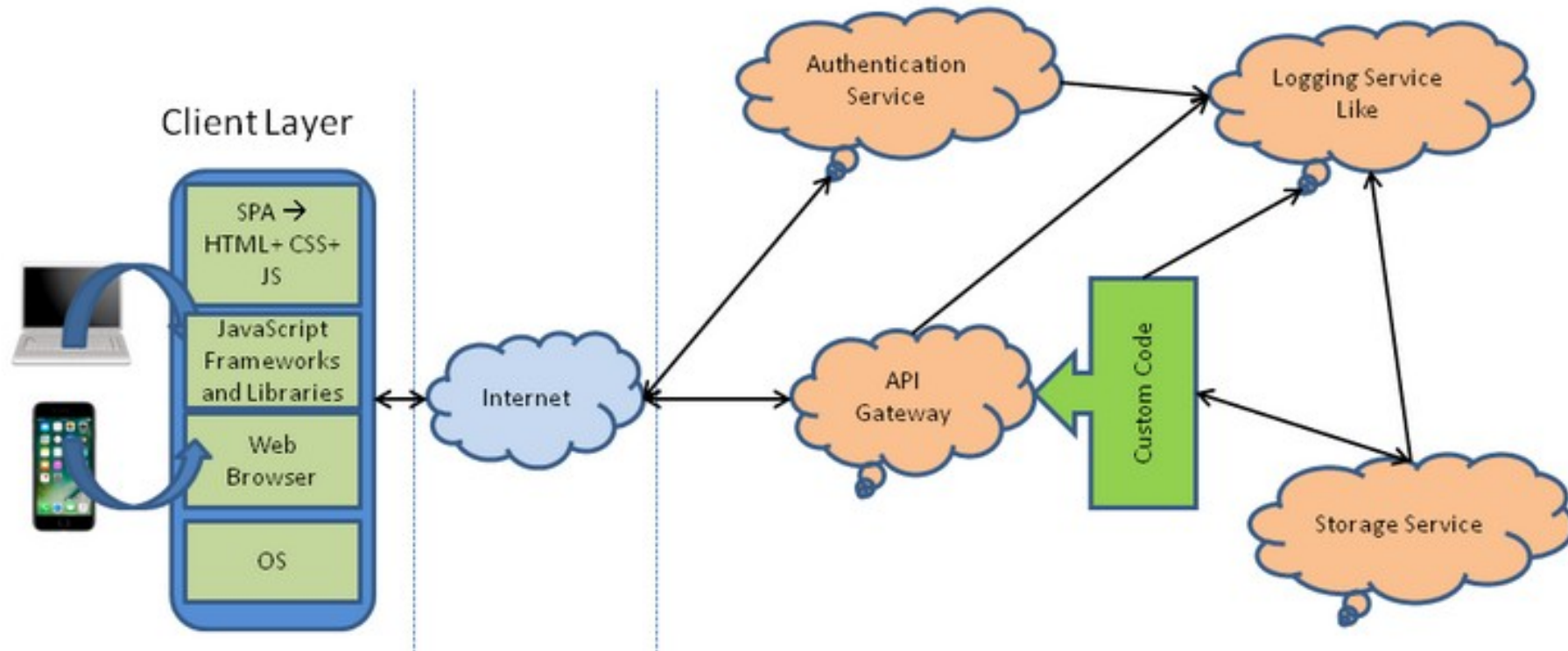
Эпоха микросервисов:





Изменения архитектуры веб-приложений

Serverless архитектура:





Современный подход к клиент-серверной коммуникации

Рассмотрим:

- REST
- GraphQL
- Брокер сообщений (RabbitMQ)
- gRPC
- SignalR (WebSockets)
- Как все это живет в одном проекте?



HTTP (REST API / RESTful)

- Representational State Transfer (REST) в переводе — это передача состояния представления. Технология позволяет получать и модифицировать данные и состояния удаленных приложений, передавая HTTP-вызовы через интернет или любую другую сеть.
- Application Programming Interface (API), или программный интерфейс приложения — это набор инструментов, который позволяет одним программам работать с другими. API предусматривает, что программы могут работать в том числе и на разных компьютерах. В этом случае требуется организовать интерфейс API так, чтобы ПО могло запрашивать функции друг друга через сеть.
- Также API должно учитывать, что программы могут быть написаны на различных языках программирования и работать в разных операционных системах.



HTTP (REST API / RESTful)

- REST API применяют везде, где пользователю сайта или веб-приложения нужно предоставить данные с сервера. Например, при нажатии иконки с видео на видеохостинге REST API проводит операции и запускает ролик с сервера в браузере. В настоящее время это самый распространенный способ организации API. Он вытеснил ранее популярные способы SOAP и WSDL.
- У RESTful нет единого стандарта работы: его называют «архитектурным стилем» для операций по работе с серверов. Такой подход в 2000 году в своей диссертации ввел программист и исследователь Рой Филдинг, один из создателей протокола HTTP.



Принципы REST API

- **Отделение клиента от сервера (Client-Server).**
 - Клиент — это пользовательский интерфейс сайта или приложения, например, поисковая строка видеохостинга. В REST API код запросов остается на стороне клиента, а код для доступа к данным — на стороне сервера.
 - Это упрощает организацию API, позволяет легко переносить пользовательский интерфейс на другую платформу и дает возможность лучше масштабировать серверное хранение данных.



Принципы REST API

- **Отсутствие записи состояния клиента (Stateless).**
 - Сервер не должен хранить информацию о состоянии (проведенных операций) клиента. Каждый запрос от клиента должен содержать только ту информацию, которая нужна для получения данных от сервера.



Принципы REST API

- **Кэшируемость (Cacheable).**
 - В данных запроса должно быть указано, нужно ли кэшировать данные (сохранять в специальном буфере для частых запросов). Если такое указание есть, клиент получит право обращаться к этому буферу при необходимости.



Принципы REST API

- **Единство интерфейса (Uniform Interface).**
 - Все данные должны запрашиваться через один URL-адрес стандартными протоколами, например, HTTP. Это упрощает архитектуру сайта или приложения и делает взаимодействие с сервером понятнее.



Принципы REST API

- **Многоуровневость системы (Layered System).**
 - В RESTful сервера могут располагаться на разных уровнях, при этом каждый сервер взаимодействует только с ближайшими уровнями и не связан запросами с другими.



Принципы REST API

- **Предоставление кода по запросу (Code on Demand).**
 - Серверы могут отправлять клиенту код (например, скрипт для запуска видео). Так общий код приложения или сайта становится сложнее только при необходимости.



Принципы REST API

- **Начало от нуля (Starting with the Null Style).**
 - Клиент знает только одну точку входа на сервер. Дальнейшие возможности по взаимодействию обеспечиваются сервером.



Стандарты

Сам по себе RESTful не является стандартом или протоколом. Разработчики руководствуются принципами REST API для создания эффективной работы с серверов для своих сайтов и приложений. Принципы позволяют выстраивать серверную архитектуру с помощью других протоколов: HTTP, URL, JSON и XML.



Стандарты

Несмотря на отсутствие стандартов, при создании REST API есть общепринятые лучшие практики, например:

- использование защищенного протокола HTTPS
- использование инструментов для разработки API Blueprint и Swagger
- применение приложения для тестирования Get Postman
- применение как можно большего количества HTTP-кодов (список)
- архивирование больших блоков данных



Архитектура

- REST API основывается на протоколе передачи гипертекста HTTP (Hypertext Transfer Protocol). Это стандартный протокол в интернете, созданный для передачи гипертекста. Сейчас с помощью HTTP отправляют любые другие типы данных.
- Каждый объект на сервере в HTTP имеет свой уникальный URL-адрес в строгом последовательном формате. Например, информация об автомобиле с id - 2 на сервере будет доступна по адресу `http://example.com/vehicles/2`.

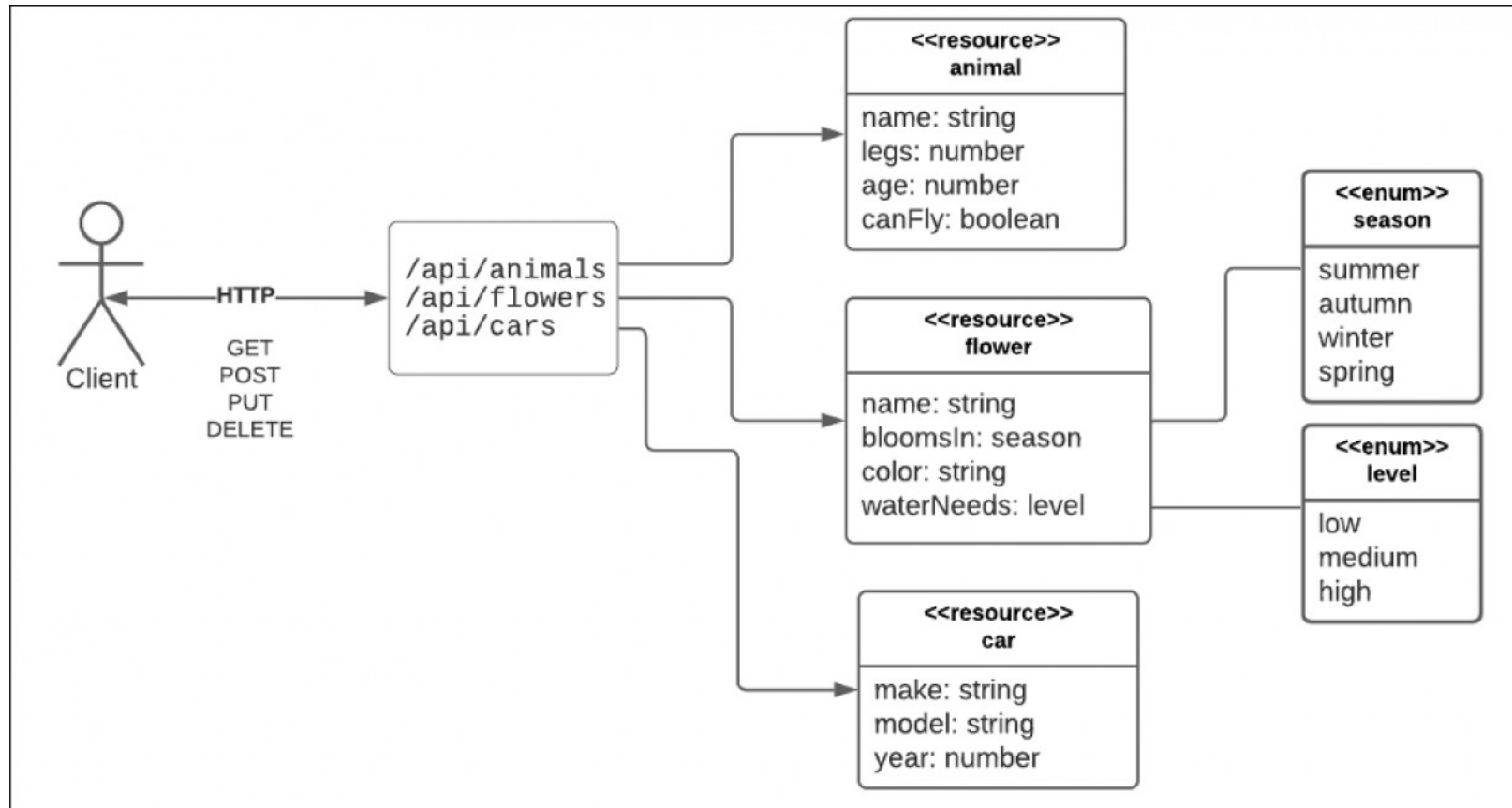


Методы REST API

- В REST API есть четыре метода HTTP, которые используют для действий с объектами на серверах:
 - GET (получение информации о данных или списка объектов)
 - DELETE (удаление данных)
 - POST (добавление или замена данных)
 - PUT (регулярное обновление данных)
- Такие запросы еще называют идентификаторами CRUD: create (создать), read (прочитать), update (обновить) delete (удалить). Это стандартный набор действий для работы с данными.



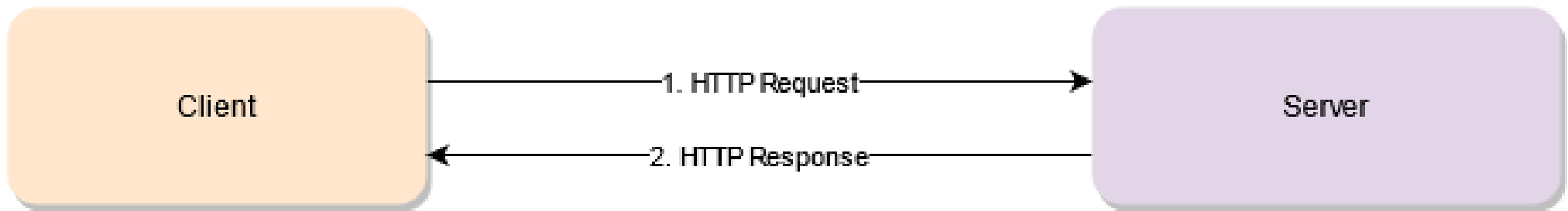
Методы REST API





Методы REST API

- Такое взаимодействие носит синхронный характер:





HTTP-статусы

- Каждый REST API запрос сообщает о результатах работы числовыми кодами — HTTP-статусами.
- Например, редактирование записи на сервере может отработать успешно (код 200), может быть заблокировано по соображениям безопасности (код 401 или 403), а то и вообще сломаться в процессе из-за ошибки сервера (код 500). Цифровые статусы выполнения ошибок — аналог пользовательских сообщений с результатами работы программы.



Форматы сообщений

REST API позволяет обмениваться не только текстовой информацией. С помощью этого инструмента можно передавать файлы и данные в специальных форматах: XML, JSON, Protobuf.



Преимущества

- Простота реализации.
- Широко используемый «стандарт».
- Надежный.
- Технологически «агностик».
- Удобное тестирование (Postman).



Недостатки

- HTTP медленный, надежность добавляет задержку.
- Таймауты (заккрытие соединения если сервер долго отвечает).
- Если сервер упал, клиент узнает не скоро 😞



Сценарии использования

- CRUD
- Resource Data-Driven сервисы



Инструменты вокруг REST API

- Тестирование REST API.
 - Postman
 - <https://insomnia.rest>
- Инструменты для описания спецификации REST API:
 - Open API (Swagger) (<https://swagger.io/>).
 - <https://stoplight.io/>
- Описание и валидации схемы REST.
 - <https://json-schema.org/>



Будут вопросы? Не стесняйтесь!

